

Wariant 1: RK4 ze stałym krokiem całkowania w którym model jest wprost wpisany w algorytm. Jest to najprostsza implementacja.

Sugestie z literatury odnośnie wartości:

Geva-Zatorsky i in., 2006 - pokazano, że poziomy p53 w zdrowych komórkach są umiarkowane i pulsują w zakresie podobnym do przyjętego poziomu 50 nM.

Lahav i in., 2004 - w badaniach nad dynamiką p53 opisali stabilne poziomy p53 i MDM2 w komórkach bez uszkodzeń DNA, co odpowiada wartościom początkowym - MDM2 ~100 nM. PTEN, jako białko supresorowe nowotworów, zazwyczaj występuje w większych stężeniach niż p53, co się zgadza z wartością 300 nM.

Zmienna	Wartość	Uzasadnienie
p53	20–40	niski na starcie, wzrasta przy uszkodzeniu DNA
mdcyto	500–1500	szybka synteza przez p53
mdmn	500–1500	transport z cytoplazmy do jądra
pten	200–1000	stabilny supresor, działa powoli
h	0.5 (minuty)	bardzo dobre do RK4 i precyzyjnego odwzorowania

Co oznaczają te konkretne wartości?

Nazwa zmiennej	Znaczenie	Ustawiona wartość	Interpretacja
value_siRNA	siRNA tłumi ekspresję MDM2	0.02	b. silna inhibicja, tylko 2% normalnej aktywności pozostało
value_PTEN_off	wyłączenie PTEN (np. mutacja)	0	całkowity brak aktywności PTEN
value_no_DNA_damage	brak uszkodzeń DNA → spadek degradacji MDM2	0.1	10% normalnej degradacji w braku sygnału uszkodzenia DNA

$h = 0.5$

$p53 = 40$

$mdcyto = 100$

$mdmn = 100$

$pten = 200$

$ile_krokow = \text{int}((48 * 60) / h)$

```

import matplotlib.pyplot as plt
import numpy as np

#parametry modelu
p1 = 8.8
p2 = 440
p3 = 100
d1 = 1.375e-14
d2 = 1.375e-4
d3 = 3e-5
k1 = 1.925e-4
k2 = 1e5
k3 = 1.5e5

value_siRNA = 0.02
value_PTEN_off = 0
value_no_DNA_damage = 0.1

#p53
def f_p53(p53, mdmn):
    wynik = p1 - d1 * p53 * (mdmn * mdmn)
    return wynik

#mdm w cytoplazmie
def f_mdmcyto(p53, mdmcyto, pten, siRNA=False,
no_DNA_damage=False):
    if siRNA == True:
        siRNA_czynnik = value_siRNA
    else:
        siRNA_czynnik = 1

    if no_DNA_damage == True:
        dna_czynnik = value_no_DNA_damage
    else:
        dna_czynnik = 1

    cz1 = p2 * siRNA_czynnik * (p53**4)
    cz2 = (p53**4) + (k2**4)
    wynik1 = cz1 / cz2

    cz3 = (k1 * (k3**2)) / ((k3**2) + (pten**2)) * mdmcyto
    cz4 = d2 * dna_czynnik * mdmcyto

    wynik = wynik1 - cz3 - cz4
    return wynik

```

```

#mdm w jądrze
def f_mdmn(mdmn, mdmcyto, pten, no_DNA_damage=False):
    if no_DNA_damage == True:
        czynnik = value_no_DNA_damage
    else:
        czynnik = 1

    cz1 = (k1 * (k3**2)) / ((k3**2) + (pten**2)) * mdmcyto
    cz2 = d2 * czynnik * mdmn

    wynik = cz1 - cz2
    return wynik

#PTEN
def f_pten(pten, p53, pten_off=False):
    if pten_off == True:
        czynnik = value_PTEN_off
    else:
        czynnik = 1

    cz1 = p3 * czynnik * (p53**4)
    cz2 = (p53**4) + (k2**4)
    wynik = cz1 / cz2 - d3 * pten
    return wynik

#RK4
def RK4const(p53, mdcyto, mdmn, pten, h, siRNA=False,
pten_off=False, no_DNA_damage=False):
    a1 = f_p53(p53, mdmn)
    b1 = f_mdmcyto(p53, mdcyto, pten, siRNA, no_DNA_damage)
    c1 = f_mdmn(mdmn, mdcyto, pten, no_DNA_damage)
    d1_pten = f_pten(pten, p53, pten_off)

    a2 = f_p53(p53 + h * a1 / 2, mdmn + h * c1 / 2)
    b2 = f_mdmcyto(p53 + h * a1 / 2, mdcyto + h * b1 / 2, pten + h
* d1_pten / 2, siRNA, no_DNA_damage)
    c2 = f_mdmn(mdmn + h * c1 / 2, mdcyto + h * b1 / 2, pten + h *
d1_pten / 2, no_DNA_damage)
    d2_pten = f_pten(pten + h * d1_pten / 2, p53 + h * a1 / 2,
pten_off)

    a3 = f_p53(p53 + h * a2 / 2, mdmn + h * c2 / 2)
    b3 = f_mdmcyto(p53 + h * a2 / 2, mdcyto + h * b2 / 2, pten + h
* d2_pten / 2, siRNA, no_DNA_damage)
    c3 = f_mdmn(mdmn + h * c2 / 2, mdcyto + h * b2 / 2, pten + h *
d2_pten / 2, no_DNA_damage)

```

```

    d3_pten = f_pten(pten + h * d2_pten / 2, p53 + h * a2 / 2,
pten_off)

    a4 = f_p53(p53 + h * a3, mdmn + h * c3)
    b4 = f_mdmcyto(p53 + h * a3, mdcyto + h * b3, pten + h *
d3_pten, siRNA, no_DNA_damage)
    c4 = f_mdmn(mdmn + h * c3, mdcyto + h * b3, pten + h * d3_pten,
no_DNA_damage)
    d4_pten = f_pten(pten + h * d3_pten, p53 + h * a3, pten_off)

    p53 = p53 + h * (a1 + 2*a2 + 2*a3 + a4) / 6
    mdcyto = mdcyto + h * (b1 + 2*b2 + 2*b3 + b4) / 6
    mdmn = mdmn + h * (c1 + 2*c2 + 2*c3 + c4) / 6
    pten = pten + h * (d1_pten + 2*d2_pten + 2*d3_pten + d4_pten) /
6

    return p53, mdcyto, mdmn, pten

def main():
    h = 0.5
    p53 = 40
    mdcyto = 100
    mdmn = 100
    pten = 200
    ile_krokow = int((48 * 60) / h)

    scenariusze = {
        "A_Podstawowy": (False, False, True),
        "B_Uszkodzenie_DNA": (False, False, False),
        "C_Nowotwor": (False, True, False),
        "D_Terapia": (True, True, False),
    }

    for nazwa, ustawienia in scenariusze.items():
        siRNA = ustawienia[0]
        pten_off = ustawienia[1]
        brak_uszkodzen = ustawienia[2]

        p53 = 50
        mdcyto = 100
        mdmn = 100
        pten = 300

        lista_czasu = []
        lista_p53 = []
        lista_mdmcyto = []
        lista_mdmn = []
        lista_pten = []

```

```

for i in range(ile_krokov):
    t = i * h
    lista_czasu.append(t)
    lista_p53.append(p53)
    lista_mdmcyto.append(mdcyto)
    lista_mdmn.append(mdmn)
    lista_pten.append(pten)

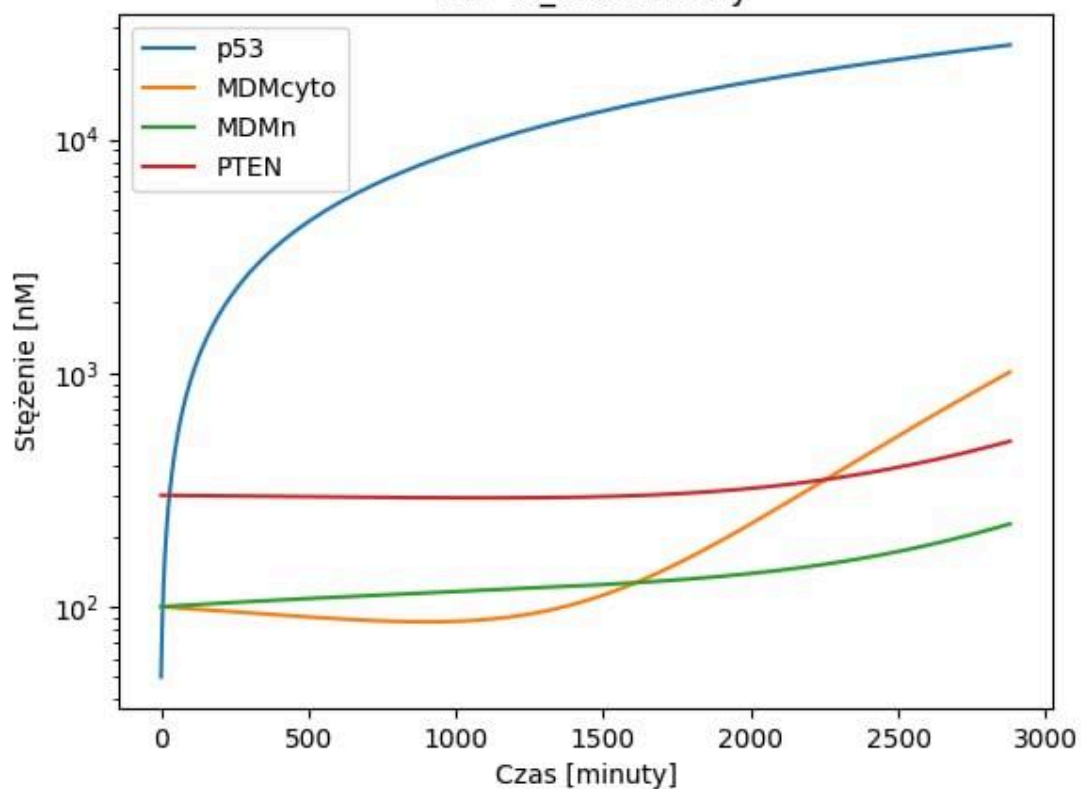
    p53, mdcyto, mdmn, pten = RK4const(p53, mdcyto, mdmn,
pten, h, siRNA, pten_off, brak_uszkodzen)

plt.plot(lista_czasu, lista_p53, label="p53")
plt.plot(lista_czasu, lista_mdmcyto, label="MDMcyto")
plt.plot(lista_czasu, lista_mdmn, label="MDMn")
plt.plot(lista_czasu, lista_pten, label="PTEN")
plt.xlabel("Czas [minuty]")
plt.ylabel("Stężenie [nM]")
plt.title("48h - " + nazwa)
plt.yscale("log")
plt.legend()
plt.show()
plt.close()

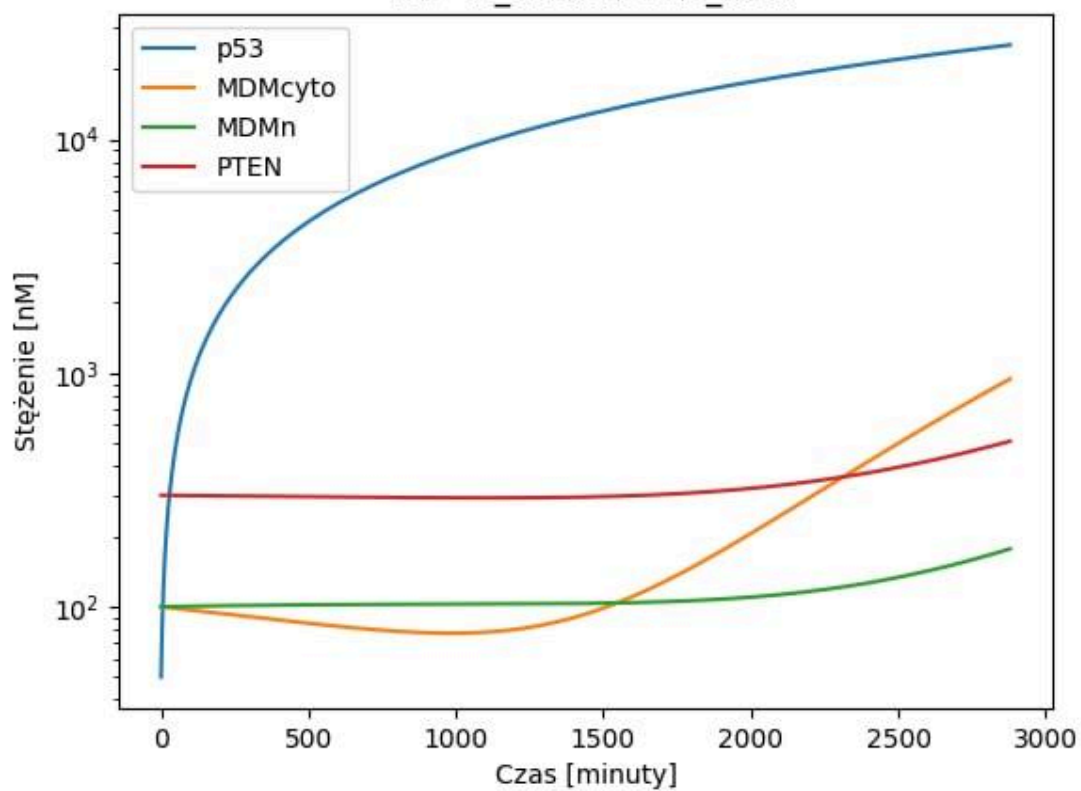
if __name__ == "__main__":
    main()

```

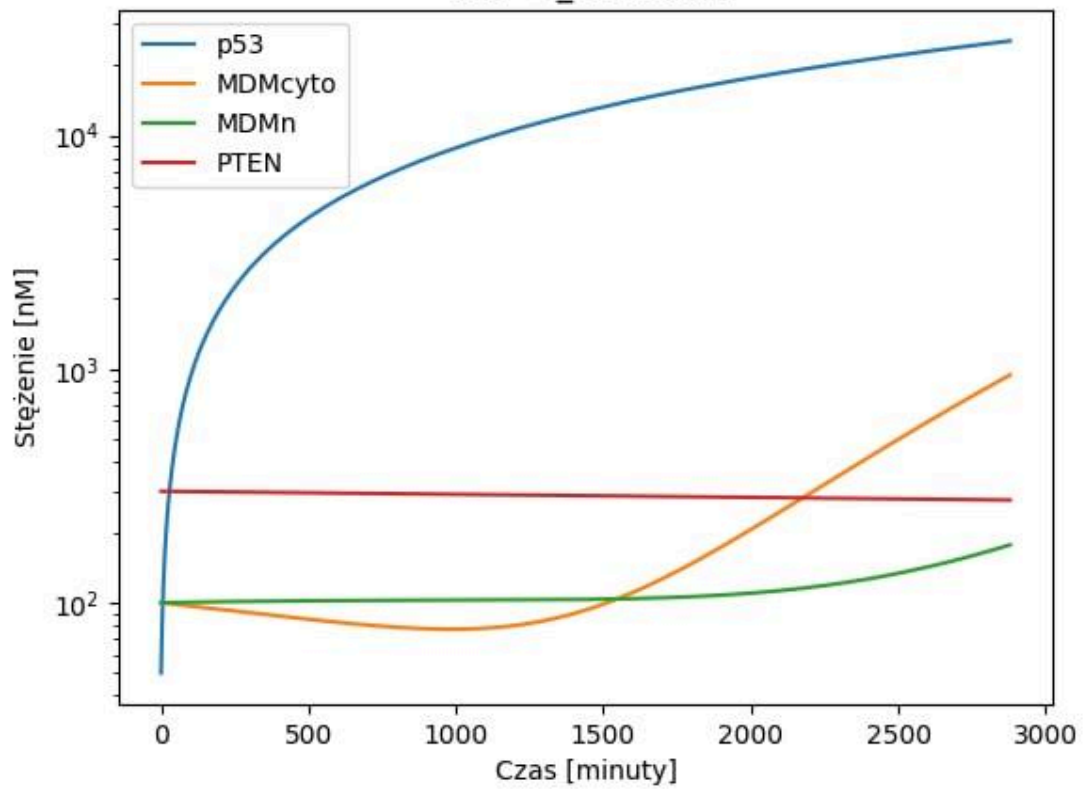
48h - A_Podstawowy



48h - B_Uszkodzenie_DNA



48h - C_Nowotwor



48h - D_Terapia

